

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO**

**ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA**

**DETECÇÃO DE VULNERABILIDADES EM CÓDIGO-FONTE COM
MACHINE LEARNING**

PLANO DE PROJETO – 1º BIMESTRE

Orientador: Prof. Julio Cesar Nievola

Visto do Orientador

**CURITIBA
2026**

RESUMO

Palavras-chave: Palavra 1. Palavra 2. Palavra 3.

SUMÁRIO

1	INTRODUÇÃO	3
1.1	MOTIVAÇÃO E RELEVÂNCIA	3
1.2	OBJETIVOS	3
1.2.1	Objetivo Geral	3
1.2.2	Objetivos Específicos	4
1.3	ESTRUTURA DO DOCUMENTO	4
2	DETALHAMENTO DO PROBLEMA	5
3	ESTADO DA ARTE	6
4	TRABALHO A SER DESENVOLVIDO	7
4.1	DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS	7
4.2	DETALHAMENTO DOS MÓDULOS FUNCIONAIS	7
5	TECNOLOGIAS QUE SERÃO UTILIZADAS	8
5.1	TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS	8
5.2	TECNOLOGIAS ALTERNATIVAS AVALIADAS	8
6	PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO	9
6.1	PROCEDIMENTOS DE TESTE	9
6.2	CRITÉRIOS DE ACEITE E DESEMPENHO	9
6.3	ANÁLISE DOS RISCOS	9
7	CRONOGRAMA DO PROJETO	10
8	CONCLUSÃO	11
	REFERÊNCIAS	12

1 INTRODUÇÃO

1.1 MOTIVAÇÃO E RELEVÂNCIA

Com o aumento do uso de Inteligência Artificial para gerar código, trechos produzidos automaticamente vêm sendo incorporados a projetos sem análise adequada de segurança, o que pode introduzir vulnerabilidades como injeção de comandos, vazamento de dados e falhas de gerenciamento de memória. Estudos recentes indicam que código assistido por IA apresenta, em média, 4,4 vezes mais vulnerabilidades que código escrito apenas por humanos, com certos tipos de falha aparecendo com maior frequência e templates inseguros se repetindo em diferentes repositórios, sugerindo padrões induzidos pelos próprios modelos. Com isso surge uma necessidade de se tomar uma medida proativa para identificar vulnerabilidades em código-fonte antes de sua integração a um sistema, de forma a reduzir o risco de exploração de falhas de segurança e proteger dados sensíveis de usuários e empresas.

Nota-se também que embora existam ferramentas de análise estática ajudam a identificar parte desses problemas elas frequentemente geram falsos positivos e possuem dificuldade para compreender o contexto do código, como a intenção do desenvolvedor e as chamadas entre funções. Essa limitação é o que torna o problema relevante do ponto de vista técnico: não basta detectar vulnerabilidades já catalogadas por regras fixas, é preciso compreender o contexto do código para reduzir os falsos positivos e priorizar corretamente os alertas mais críticos.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver um software capaz de analisar novos trechos de código-fonte antes de sua integração a um sistema, identificando padrões e erros comuns que possam resultar em vulnerabilidades de segurança conhecidas, utilizando técnicas de Machine Learning e Large Language Models para tornar essa análise mais contextual do que a oferecida pelas ferramentas tradicionais de análise estática.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Realizar uma revisão bibliográfica sobre vulnerabilidades de segurança em código-fonte, ferramentas de análise estática e técnicas de Machine Learning e Large Language Models aplicadas à detecção de vulnerabilidades;
- Definir a arquitetura do sistema, incluindo a decomposição em módulos e o diagrama em blocos do software;
- Selecionar as tecnologias que serão utilizadas para o desenvolvimento do software, justificando suas escolhas;
- Implementar o software, integrando os módulos e garantindo sua funcionalidade;
- Realizar testes e validação do software, avaliando sua eficácia na detecção de vulnerabilidades em código-fonte.

1.3 ESTRUTURA DO DOCUMENTO

Este documento está estruturado da seguinte forma: a Seção 2 detalha o problema a ser resolvido; a Seção 3 apresenta o estado da arte das soluções existentes; a Seção 4 descreve o trabalho a ser desenvolvido, incluindo a decomposição em módulos e o diagrama em blocos do sistema; a Seção 5 apresenta as tecnologias que serão utilizadas; a Seção 6 descreve os procedimentos de teste e validação; a Seção 6.3 traz a análise dos riscos do projeto; a Seção 7 apresenta o cronograma; e a Seção 8 conclui o documento.

2 DETALHAMENTO DO PROBLEMA

Primeiramente para entender os problemas de segurança em código, precisamos definir a produção de código antes das ferramentas de Inteligência Artificial. O código é produzido por desenvolvedores humanos, que escrevem manualmente cada linha de código, utilizando suas habilidades e conhecimentos para criar software funcional. Esse processo envolve a compreensão dos requisitos do sistema, a implementação de algoritmos e estruturas de dados, e a realização de testes para garantir que o código funcione corretamente. No entanto, mesmo com desenvolvedores experientes, erros podem ocorrer, resultando em vulnerabilidades de segurança.

No entanto, estamos vivendo uma verdadeira revolução na produção de código com o advento de ferramentas de Inteligência Artificial, como Claude Code, Codex, Antigravity, Cursor e o GitHub Copilot, que utilizam modelos de linguagem avançados para auxiliar os desenvolvedores na escrita de código. Essas ferramentas podem gerar código automaticamente, sejam por trechos ou até mesmo abordagens como o uso de agentes para criar tudo através com base em descrições em linguagem natural. Embora essas ferramentas possam aumentar a produtividade e reduzir o tempo de desenvolvimento, elas também introduzem desafios de segurança.

Com esse advento, surgem preocupações sobre a qualidade e segurança do código gerado automaticamente. Estudos recentes indicam que código assistido por IA apresenta, em média, 4,4 vezes mais vulnerabilidades que código escrito apenas por humanos, com certos tipos de falha aparecendo com maior frequência e templates inseguros se repetindo em diferentes repositórios, sugerindo padrões induzidos pelos próprios modelos (State of Cyber; Wu *et al.*, 2026, 2025). Embora ferramentas de análise estática ajudem a identificar esses problemas, elas frequentemente geram falsos positivos e possuem dificuldade para compreender o contexto do código, como a intenção do desenvolvedor e chamadas entre funções (Yang *et al.*; TechScience, 2025, 2025). Por isso, há demanda por abordagens mais contextuais, baseadas em Machine Learning e Large Language Models, que vêm sendo aplicadas à detecção de vulnerabilidades e à priorização de alertas em código gerado automaticamente (Wang *et al.*; MDPI, 2025, 2026).

3 ESTADO DA ARTE

4 TRABALHO A SER DESENVOLVIDO

Objetivo : DESENVOLVER um software que analise...

4.1 DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS

4.2 DETALHAMENTO DOS MÓDULOS FUNCIONAIS

5 TECNOLOGIAS QUE SERÃO UTILIZADAS

5.1 TECNOLOGIAS ESCOLHIDAS E JUSTIFICATIVAS

5.2 TECNOLOGIAS ALTERNATIVAS AVALIADAS

6 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

6.1 PROCEDIMENTOS DE TESTE

6.2 CRITÉRIOS DE ACEITE E DESEMPENHO

6.3 ANÁLISE DOS RISCOS

Risco	Probab.	Impacto	Mitigação
Escassez ou desbalan- ceamento de dados ro- tulos de vulnerabili- dades	Média	Alto	Utilizar conjuntos de dados públicos já consolidados e técnicas de balanceamento de classes; complementar com dados sintéticos se necessário
Baixo desempenho do modelo de Machine Le- arning	Média	Alto	Validação cruzada, separação clara entre treino/teste e ajuste incremental dos modelos
Complexidade na inte- gração de Large Lan- guage Models ao <i>pipe- line</i> de análise	Média	Médio	Tratar o uso de LLMs como módulo complementar e opcional, priorizando primeiro um classificador de ML mais simples e funcional
Restrição de tempo do cronograma acadêmico	Alta	Alto	Priorizar entregas incrementais, com um MVP funcional antes de adicionar funcionalidades complementares
Limitações de poder computacional dos integrantes para treinamento de modelos	Baixa	Médio	Uso de infraestrutura em nuvem gratuita/educacional caso necessário
Falsos positivos elevados prejudicando a confiança na ferramenta	Média	Alto	Comparação contínua com ferramentas de análise estática tradicionais e ajuste dos limiares de classificação

7 CRONOGRAMA DO PROJETO

8 CONCLUSÃO

REFERÊNCIAS

MDPI. A survey on large language models in software security: Opportunities and threats. **Computers**, v. 15, n. 4, 2026. Disponível em: <https://www.mdpi.com/2073-431X/15/4/226>. Acesso em: 01 set. 2026.

State of Cyber. **AI-Assisted Code Introduces 4.4x More Security Vulnerabilities Than Human-Written Code, New Research Finds**. 2026. The Tennessean. Disponível em: <https://www.tennessean.com/press-release/story/223693/ai-assisted-code-introduces-4-4x-more-security-vulnerabilities-than-human-written-code-new-research-finds/>. Acesso em: 01 set. 2026.

TechScience. Static analysis techniques for secure software: A systematic review. **Journal of Cyber Security**, v. 7, n. 1, 2025. Disponível em: <https://www.techscience.com/JCS/v7n1/64045/html>. Acesso em: 01 set. 2026.

WANG, Y. *et al.* A generative ai cybersecurity risks mitigation model for code generation: Using ann-ism hybrid approach. **Scientific Reports**, 2025. Disponível em: <https://www.nature.com/articles/s41598-025-34350-3>. Acesso em: 01 set. 2026.

WU, Z. *et al.* **AI Code in the Wild: Measuring Security Risks and Mitigations**. 2025. ArXiv:2512.18567. Disponível em: <https://arxiv.org/abs/2512.18567>. Acesso em: 01 set. 2026.

YANG, X. *et al.* **ZeroFalse: Improving Precision in Static Analysis with LLMs**. 2025. ArXiv:2510.02534. Disponível em: <https://arxiv.org/abs/2510.02534>. Acesso em: 01 set. 2026.